

FairSplit+ Database Schema in SQL

Beschrijving van het databaseschema van FairSplit+, inclusief tabellen, relaties en constraints. Het schema is opgezet in **PostgreSQL (Supabase)** en vormt de basis voor gebruikers, groepen, uitgaven en afrekeningen.

- Row Level Security (RLS) is ingeschakeld voor alle tabellen.
 - Er is een development policy actief die alle acties toestaat.
 - Het schema is handmatig aangemaakt via SQL in Supabase.
-

Overzicht Tables

- **users** – Gebruikers van de applicatie
- **groups** – Groepen waarin kosten worden gedeeld
- **group_members** – Koppelt gebruikers aan groepen
- **expenses** – Uitgaven binnen een groep
- **expense_items** – Detailregels per uitgave (voor toekomstige functionaliteit)
- **expense_shares** – Verdeling van kosten per gebruiker
- **payments** – Werkelijke betalingen tussen gebruikers

1. USERS

Bevat basisinformatie over gebruikers.

```
create table public.users (  
  user_id serial not null,  
  name character varying(100) not null,  
  phone_number character varying(30) null,  
  payment_method character varying(50) null,  
  
  constraint users_pkey primary key (user_id)  
);
```

Opmerkingen:

- payment_method wordt gebruikt voor IBAN-nummer.
- Elke gebruiker heeft een uniek user_id.

2. GROUPS

Definieert een groep waarin kosten worden gedeeld.

```
create table public.groups (  
    group_id serial not null,  
    name character varying(100) not null,  
    start_date date not null,  
    end_date date not null,  
    organizer_id integer not null,  
    app_fee numeric(10, 2) null default 0.00,  
    icon text null,  
  
    constraint groups_pkey primary key (group_id),  
    constraint groups_organizer_id_fkey foreign KEY (organizer_id) references users  
(user_id)  
) TABLESPACE pg_default;
```

Opmerkingen:

- Elke groep heeft exact één organisator.
 - app_fee krijgt momenteel via create_group een standaard waarde 3, maar kan bij verdere ontwikkeling een eventueel specifieke waarde in deze table krijgen per groep/naargelang abonnement.
-

3. GROUP_MEMBERS

Koppelt gebruikers aan groepen en bepaalt hun rol.

```
create table public.group_members (  
    group_id integer not null,  
    user_id integer not null,  
    role character varying(20) null default 'member'::character varying,  
    joined_date date null default CURRENT_DATE,  
  
    constraint group_members_pkey primary key (group_id, user_id),  
    constraint group_members_group_id_fkey foreign KEY (group_id) references groups  
(group_id) on delete CASCADE,  
    constraint group_members_user_id_fkey foreign KEY (user_id) references users  
(user_id) on delete CASCADE  
) TABLESPACE pg_default;
```

Opmerkingen:

- Samengestelde primaire sleutel voorkomt dubbele leden.

- Cascade delete zorgt voor automatische opschoning bij verwijderen van een groep of gebruiker.

4. EXPENSES

Registreert uitgaven binnen een groep.

```
create table public.expenses (
    expense_id serial not null,
    group_id integer not null,
    paid_by integer not null,
    description text null,
    total_amount numeric(10, 2) not null,
    receipt_image text null,

    constraint expenses_pkey primary key (expense_id),
    constraint expenses_group_id_fkey foreign KEY (group_id) references groups
(group_id) on delete CASCADE,
    constraint expenses_paid_by_fkey foreign KEY (paid_by) references users (user_id)
) TABLESPACE pg_default;
```

Opmerkingen:

- paid_by is de persoon die het bedrag heeft voorgesloten.
- Kostenverdeling gebeurt via expense_shares.
- receipt_image kan gebruikt worden voor toekomstige functie waarbij kas-ticktjes ingescanned kunnen worden.

5. EXPENSE_ITEMS

Net zoals receipt_image is deze table voor het extraheren van line items uit kas-ticktjes. (Momenteel niet nog geen functie in de code)

```
create table public.expense_items (
    item_id serial not null,
    expense_id integer not null,
    description text not null,
    quantity integer null default 1,
    unit_price numeric(10, 2) not null,
    total numeric generated always as (quantity * unit_price) stored,

    constraint expense_items_pkey primary key (item_id),
    constraint expense_items_expense_id_fkey foreign key (expense_id) references
expenses (expense_id) on delete cascade
);
```

Opmerkingen:

- Voor het opsplitsen van uitgaven in afzonderlijke items.
- ON DELETE CASCADE zorgt dat items verdwijnen als de bijbehorende expense wordt verwijderd.

6. EXPENSE_SHARES

Geeft aan welk deel van een uitgave iedere gebruiker verschuldigd is.

```
create table public.expense_shares (
    expense_id integer not null,
    user_id integer not null,
    amount numeric(10, 2) not null,

    constraint expense_shares_pkey primary key (expense_id, user_id),
    constraint expense_shares_expense_id_fkey foreign key (expense_id) references
expenses (expense_id) on delete cascade,
    constraint expense_shares_user_id_fkey foreign key (user_id) references users
(user_id) on delete cascade,
    constraint expense_shares_amount_check check (amount >= 0)
);
```

Opmerkingen:

- Zorgt ervoor dat kosten nooit negatief kunnen zijn.
- Essentieel voor saldo- en afrekenlogica.

7. PAYMENTS

Registreert daadwerkelijke betalingen tussen gebruikers.

```
create table public.payments (
    payment_id serial not null,
    from_user integer not null,
    to_user integer not null,
    group_id integer null,
    amount numeric(10, 2) not null,
    method character varying(30) null,
    status character varying(20) null default 'pending',
    payment_date date null default CURRENT_DATE,
    reference text null,

    constraint payments_pkey primary key (payment_id),
    constraint payments_from_user_fkey foreign key (from_user) references users
(user_id),
    constraint payments_to_user_fkey foreign key (to_user) references users (user_id),
```

```
constraint payments_group_id_fkey foreign key (group_id) references groups
(group_id),
constraint payments_amount_check check (amount > 0),
constraint payments_status_check check (
    status in ('pending', 'completed', 'failed')
)
);
```

Opmerkingen:

- Wordt gebruikt voor echte afrekeningen.
- method en reference zijn voorbereid voor eventuele uitbreidingen, maar standaard gebruiken binnen de MVP bankbetalingen.
- Betalingen beïnvloeden de uiteindelijke saldi, maar vervangen geen expenses.